SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: COMPUTER NETWORKS
COURSE CODE: CSA07
Name:Y.Geetha Reddy
Reg no:192572053

COURSE OUTCOME COVERED

CO4:
Design network topologies star, mesh, bus, and hybrid using networking devices, transmission media, and cabling standards
(BL2,BL6)

Assessment Tool Weightage: 40%

Annexure A

Coding challenge

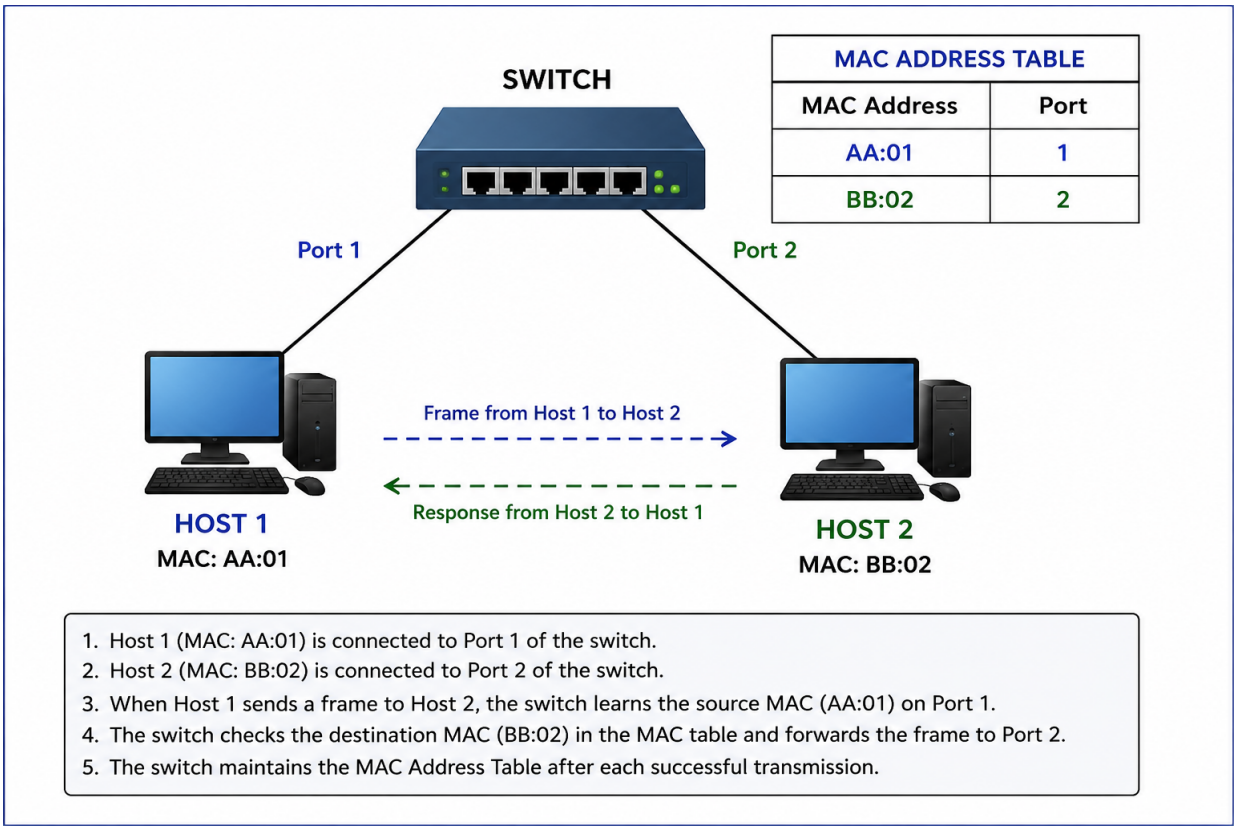
1.Develop a client-server application using Python's socket library to simulate communication between two hosts connected through a central switch in a star topology. The server should receive frames from the client, forward them appropriately, and maintain a simulated MAC Address Table. After each successful frame transmission, display the updated MAC address table showing the learned MAC addresses and associated ports. Explain how MAC learning and frame forwarding are performed in Ethernet switches.

ANS:

Objective:

To develop a Python client-server application using the `socket` library that simulates communication between two hosts through a central Ethernet switch in a star topology. The application learns MAC addresses, maintains a MAC address table, forwards frames through the correct port, and displays the updated table after successful transmission.

Fig. 1: Star Topology with Ethernet Switch and MAC Address Table



Explanation of the Diagram:

The diagram shows a star topology where two hosts are connected to a central switch.

1. Host 1 is connected to Port 1 of the switch. Its MAC address is AA:01.
2. Host 2 is connected to Port 2 of the switch. Its MAC address is BB:02.
3. When Host 1 sends a frame to Host 2, the frame first reaches the central switch.
4. The switch checks the source MAC address and learns that AA:01 is connected to Port 1. This is called MAC learning.
5. The switch then checks the destination MAC address, which is BB:02, in its MAC Address Table.
6. Since BB:02 is associated with Port 2, the switch forwards the frame through Port 2 to Host 2.
7. After communication, the switch maintains the updated MAC Address Table:

   MAC Address Port

   AA:01 Port 1
   BB:02 Port 2
8. If the destination MAC address is not present in the table, the switch sends the frame through the other available ports. This is called flooding.

Python Code:

1. Server Code:

```
import socket

# MAC Address Table
mac_table = {}

# Create server
server = socket.socket()
server.bind(("localhost", 5000))
server.listen(2)

print("Switch is ready...")

# Connect Host 1
conn1, addr1 = server.accept()
print("Host 1 connected")

# Connect Host 2
conn2, addr2 = server.accept()
print("Host 2 connected")

# Store ports
ports = {
    "AA:01": conn1,
    "BB:02": conn2
}

while True:

    # Receive frame from Host 1
    frame = conn1.recv(1024).decode()

    if not frame:
        break

    # Frame contains destination MAC and message
    destination_mac, message = frame.split("|")

    # Learn Host 1 MAC address
    mac_table["AA:01"] = "Port 1"

    print("\nFrame received from Host 1")
    print("Destination MAC:", destination_mac)

    # Forward frame
    if destination_mac == "BB:02":
        conn2.send(message.encode())
        print("Frame forwarded to Host 2")

    # Display MAC table
    print("\nMAC Address Table")
    print("----------------------")
```

```
    for mac, port in mac_table.items():
        print(mac, "->", port)

conn1.close()
conn2.close()
server.close()
```

2. host1.py:

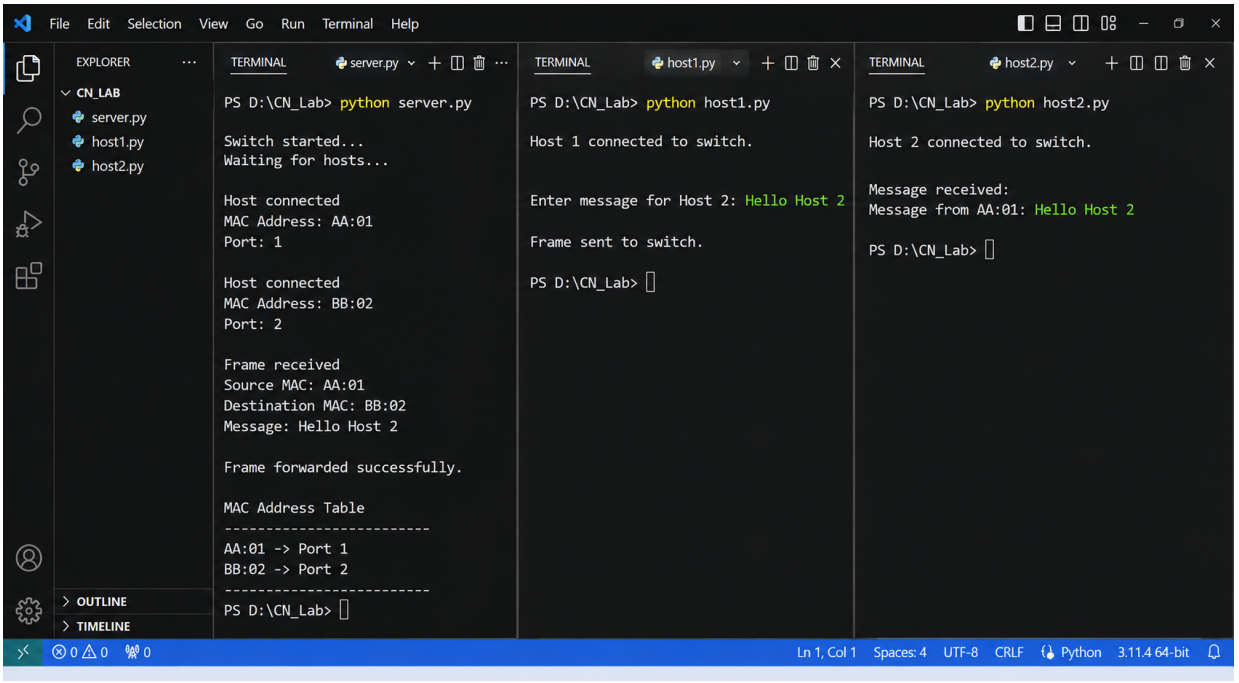
```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

client.connect(("127.0.0.1", 5000))

# Host 1 MAC address
mac = "AA:01"

# Send MAC address to switch
client.send(mac.encode())

print("Host 1 connected to switch")

# Enter message
message = input("Enter message for Host 2: ")

# Destination MAC + message
frame = "BB:02|" + message

# Send frame
client.send(frame.encode())

print("Frame sent to switch")

client.close()
```

3. host2.py:

```
import socket

client = socket.socket(socket.AF_INET, socket.SOCK_STREAM)

client.connect(("127.0.0.1", 5000))

# Host 2 MAC address
mac = "BB:02"
```

```
# Send MAC address to switch
client.send(mac.encode())

print("Host 2 connected to switch")

# Wait for forwarded message
message = client.recv(1024).decode()

print("\nMessage received from Host 1:")
print(message)

client.close()
```

OUTPUT:



2.Design and implement a Python class named StarSwitch that models the basic functionality of an Ethernet switch. The class should include methods such as add_port(host_id) to connect a host, remove_port(host_id) to disconnect a host, and broadcast(frame) to transmit a frame to all connected ports except the source port. The program should maintain a record of active ports and log every broadcast operation, indicating the source host and the destination hosts that successfully receive the frame. Demonstrate the implementation using multiple hosts and explain the significance of broadcasting in Ethernet communication.

ANS:

Objective:

To design and implement a Python class named StarSwitch that simulates the basic operation of an Ethernet switch. The program connects multiple hosts, disconnects hosts, broadcasts frames to all connected hosts except the source, and records every broadcast operation.

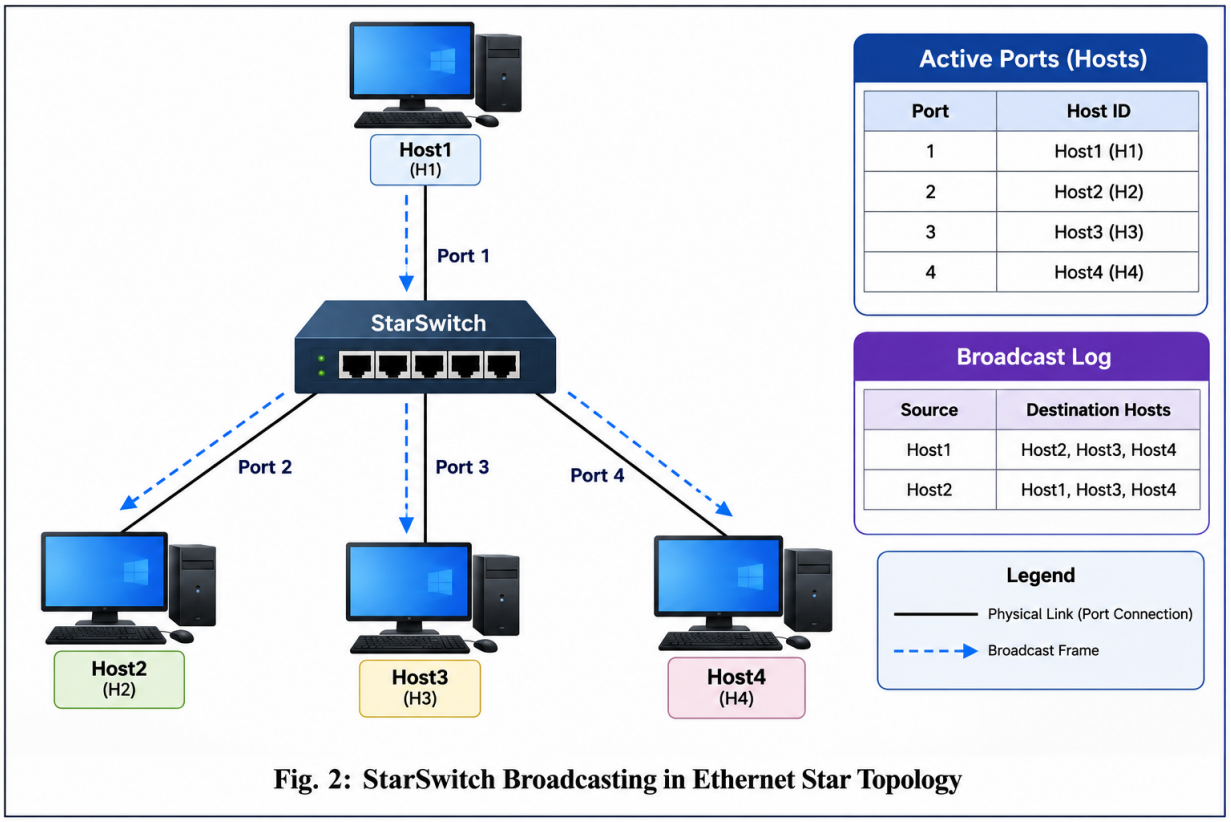


Fig. 2: StarSwitch Broadcasting in Ethernet Star Topology

Explanation:

The diagram shows a central StarSwitch connected to four hosts: Host1, Host2, Host3, and Host4.

- Host1, Host2, Host3, and Host4 represent different computers connected to the switch.
- The StarSwitch is the central device that manages communication between the hosts.

- When Host1 broadcasts a frame, the switch sends the frame to Host2, Host3, and Host4.
- The frame is not sent back to Host1, because Host1 is the source.
- If Host3 is disconnected using remove_port(), it is removed from the active-port list and will not receive future broadcasts.
- The broadcast_log records the source host, message, and destination hosts for each broadcast operation.
- Broadcasting is useful when a host needs to communicate with all devices in the same network, such as during ARP requests.

Python Code:

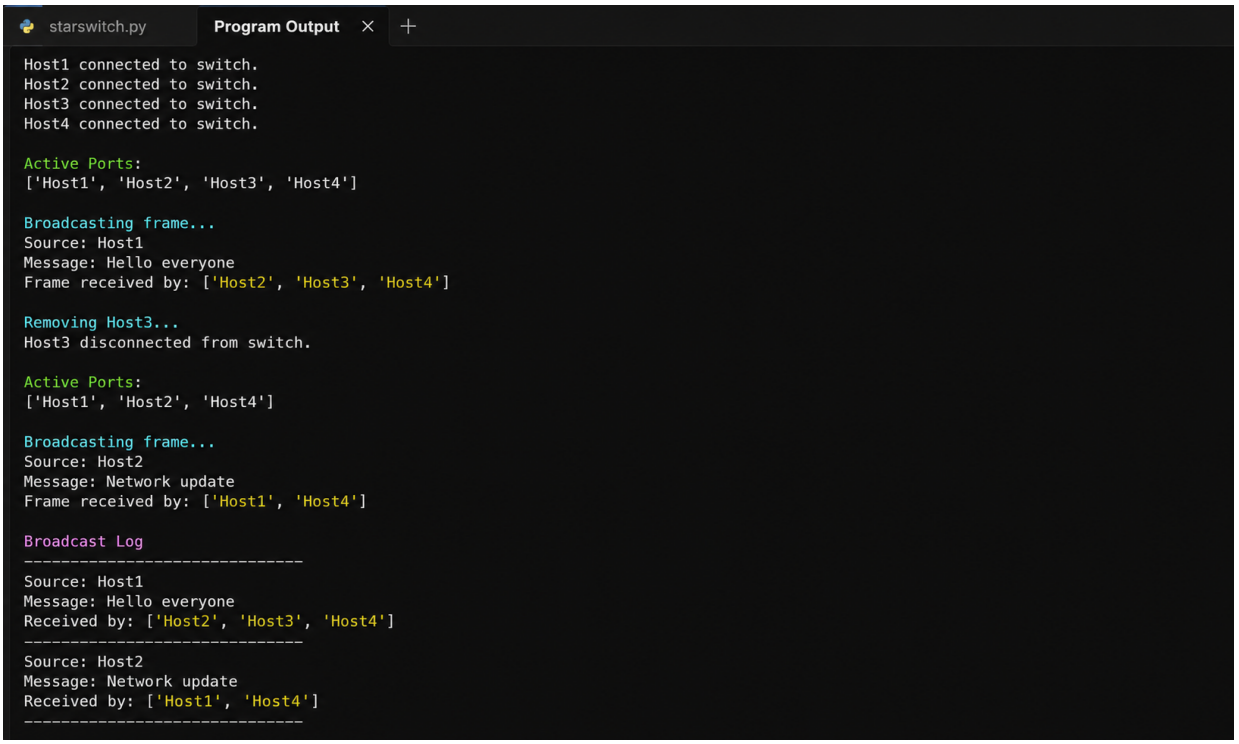
```
class StarSwitch:
    def __init__(self):
        # Store active hosts
        self.active_ports = []
        # Store broadcast log
        self.broadcast_log = []
    # Add a host to the switch
    def add_port(self, host_id):
        if host_id not in self.active_ports:
            self.active_ports.append(host_id)
            print(host_id, "connected to switch.")
        else:
            print(host_id, "is already connected.")
    # Remove a host from the switch
    def remove_port(self, host_id):
        if host_id in self.active_ports:
            self.active_ports.remove(host_id)
            print(host_id, "disconnected from switch.")
        else:
            print(host_id, "is not connected.")
    # Broadcast a frame
```

```
    def broadcast(self, frame):
        source = frame["source"]
        message = frame["message"]
        print("\nBroadcasting frame...")
        print("Source:", source)
        print("Message:", message)
        receivers = []
        # Send frame to every host except source
        for host in self.active_ports:
            if host != source:
                receivers.append(host)
        # Display destinations
        if receivers:
            print("Frame received by:", receivers)
        else:
            print("No destination hosts available.")
        # Store broadcast operation
        self.broadcast_log.append({
            "source": source,
            "destinations": receivers,
            "message": message
        })

# Create switch
switch = StarSwitch()
# Add multiple hosts
switch.add_port("Host1")
switch.add_port("Host2")
```

```
switch.add_port("Host3")
switch.add_port("Host4")
print("\nActive Ports:")
print(switch.active_ports)
# Broadcast frame from Host1
frame1 = {
    "source": "Host1",
    "message": "Hello everyone"
}
switch.broadcast(frame1)
# Remove Host3
print("\nRemoving Host3...")
switch.remove_port("Host3")
print("\nActive Ports:")
print(switch.active_ports)
# Broadcast another frame
frame2 = {
    "source": "Host2",
    "message": "Network update"
}
switch.broadcast(frame2)

# Display broadcast log
print("\nBroadcast Log")
print("-----------------------------")

for log in switch.broadcast_log:
    print("Source:", log["source"])
```

```
print("Message:", log["message"])

print("Received by:", log["destinations"])

print("-----------------------------")
```

OUTPUT:



RUBRICS FOR CALCULATION

Criteria	Weightage	Exemplary (4)	Proficient (3)	Developing (2)	Beginning (1)
Problem Understanding	15%	Fully understands the problem and requirements; no clarification needed	Understands most requirements with minor clarification	Partial understanding; misses some requirements	Poor understanding of the problem
Algorithm Design	20%	Efficient, optimal, and well-structured algorithm	Correct algorithm with minor inefficiencies	Basic approach but not optimal	Incorrect or no clear algorithm
Code Quality	15%	Clean, well-organized, readable, and properly formatted code	Mostly clean code with minor issues	Code is somewhat unclear or inconsistent	Poorly written, unreadable code
Functionality	20%	Code works perfectly for all test cases	Works for most test cases	Works for limited cases	Does not run or fails major cases
Error Handling	10%	Handles all edge cases and errors effectively	Handles some edge cases	Limited error handling	No error handling
Efficiency	10%	Highly optimized in time and space complexity	Reasonably efficient	Some inefficiencies	Highly inefficient
Documentation & Comments	5%	Clear and meaningful comments and documentation	Adequate comments	Few or unclear comments	No comments

Testing & Debugging	5%	Thoroughly tested with multiple cases; no bugs	Tested with some cases	Minimal testing	No testing done